Explain how social media connections can be represented as a graph.
How would you identify influential users? Describe one approach.

Corrected 20-Day Syllabus Master Schedule

1.Days 1 to 4: Foundations & Core Concepts: Covering initial units of DSA, OS, Architecture, SE, and Research.

How would you represent users and their connections in a social media as a graph? What type of graph would you use? How would you recommend users based on connections? Describe one approach.

- **DAY 1**
  - 05:00–08:00 **Data Structure and Algorithm Design**
    - *1. Introduction to Basic Data Structures and Algorithm Design*
    - 1.1. Basic Data Structures: Arrays, Stack, Queue, Linked list, Tree and Graph
    - 1.2. Divide and Conquer, Greedy Algorithm, Backtracking Algorithms, Branch & Bound and Randomized Algorithms
    - 1.3. Time Complexity, Space Complexity and Asymptotic notations: Big O, Big Theta and Big Omega
    -

Define circular queue. Write an algorithm to insert a new node at the end of doubly linked list.

Difference between adjacency matrix and adjacency list

How does the Greedy Algorithm works? Construct an AVL tree for following sequence of elements: 5,10,15, 30, 25, 20, 4, 2,1.

Define the growth rate of an algorithm. Explain how the cost of searching an item in a sorted array using binary search is logarithmic function of input size n (i.e. O(logzn)) [SN]

a) Explain the concept of asymptotic notations (Big O, Big Theta, and Big Omega) with examples. How are they used to analyze the time complexity of algorithms?

  - 14:00–16:00 **REVISION**
    - No +4/+7 revision exists yet. Review difficult portions of Day 1 morning, consolidate notes, write core definitions.
  - 16:30–19:00 **Operating System Design and Implementation**
    - *1. Operating System Architecture*
    - 1.1. Review on operating system: OS concepts, System Calls, Process, Threads, CPU Scheduling, Memory, Storage, I/O Management.
    - 1.2. Operating System Design Goals and Principles
    - 1.3. Operating System Structures and Kernel Types: Layered, Monolithic, Microkernel, Hybrid Kernel, Exokernel, Nanokernel, and Modular Kernel
    -

Compare and contrast monolithic, microkernel, and hybrid kernel architectures. Discuss the advantages and disadvantages of each type. Provide examples of operating systems that use these kernel architectures and explain why they are suitable for their respective use cases.

Discuss Flynn's classification in detail with its pros and cons.

Justify and derive necessary relation for the statement: “The performance improvement to be gained from using some faster mode of execution is limited by the fraction of the time the faster mode can be used.”

  - 21:00–22:00 **SAME-DAY REVISION & ACTIVE RECALL**
    - Revise: DSA 1.1, 1.2, 1.3 & OS 1.1, 1.2, 1.3. Close notes for the last 20 minutes and recall.

Explain with example Amdahl's law for measuring speed up performance of parallel systems. Consider an enhancement to processor of web server. The new CPU is 20 times faster on search queries than old processor. The old processor is busy with search queries 70% of the time, what is the speed up gained by integrating the enhanced CPU?

What is parallel processing? Why is the demand of parallel processing increasing day by day? Explain with examples.

- **DAY 2**
  - 05:00–08:00 **Advanced Computer Architecture**
    - *1. Introduction to Parallel Processing*
    - 1.1. Parallel Computer Structures
    - 1.2. Architectural Classification Parallel Processing Applications
    - 1.3. Need of parallel programming, characteristics of parallel algorithms
    - 1.4. parallel programming techniques
    - 1.5. parallel programming models
    - 1.6. parallel algorithms for multiprocessors
    - 1.7. performance of parallel algorithms
    - 1.8. parallel programming languages
    - 1.9. Problems based on parallel algorithms

Implicit vs. explicit parallel programming

Is it possible to implicitly reduce the size of an instruction? Explain your reasoning.

Synchronized vs. asynchronized parallel algorithms

Loosely Coupled Systems

What is the need of parallel programming? Explain the required characteristics of parallel algorithms.

Explain the architectural classification of computer systems based on Flynn's taxonomy. Provide examples for each category and discuss their relevance in parallel computing.

What is the message passing programming? In which case is it suitable? Explain.

Is there any form of parallelism in a uniprocessor system? If yes, explain the different forms of parallelism used in a uniprocessor system and if no, explain why it is not feasible.

In which architecture is the message passing programming model used? Compare the message passing programming model with the shared memory programming model.

What are the different mechanisms of parallelism used in a uniprocessor system? Explain.

In the example of adding 'n' numbers using 'p' processing elements, can granularity be achieved in a cost-optimal manner? Provide justification for your answer.

Explain the 6 Ps of Research. Illustrate how each stage contributes to the success of a research study with examples from IT research.

-
  -
  - 14:00–16:00 **REVISION**
    - Revise Day 1 DSA (1.1–1.3) and Day 1 OS (1.1–1.3).
  -
  - 16:30–19:00 **Research Techniques**
    - *1. Introduction to Research*
      - 1.1. Research: Objectives and Characteristics
      - 1.2. Types of Research
      - 1.3. The 6 P's of Research
      - 1.4. Research Process
      - 1.5. Research in Computer Science and Engineering

How is research defined and conceptualized within the field of computer engineering? Explain model of the research process.

"Research is the systematic approach to obtaining and confirming new and reliable knowledge." Justify this statement and explain the objectives and characteristics of effective research

Describe briefly with examples the role of "6 P's of Research" in shaping a research project.

An e-commerce platform aims to launch rapidly with a set of basic features and progressively introduce advanced functionalities over time. Explain the suitable software development process that can be implemented in the development of the project.

Why does the traditional linear software development process (like the Waterfall model) often need to be replaced or adapted? Explain the approach of execution of framework activities in Spiral model.

  -
  - 21:00–22:00 **SAME-DAY REVISION & ACTIVE RECALL**
    - Revise: Architecture 1.1–1.9 & Research 1.1–1.5.

As Software Engineering a true engineering discipline? Justify your answer by comparing its methodologies, principles, and professional standards with those of traditional engineering fields.

- **DAY 3**
  - 05:00–08:00 **Software Engineering** *(Mapped directly to SE Syllabus)*
    - *1. Introduction*
      - 1.1 Software Engineering Concepts: Participants and Roles in Software Projects, Systems and Models, Work Products, Activities, Tasks, and Resources, Notations, Methods, and Methodologies
      - 1.2 Overview of Software Process Models: Traditional Models: Waterfall, Iterative, Spiral, Agile Models, Comparison of Process Models
      - 1.3 Introduction to Requirements Engineering: Functional Requirements, Non-functional Requirements, Importance of Requirements Engineering

How do these five process models-Waterfall, Iterative, Agile, Spiral, and DevOps/CI-CD-compare in terms of their ability to handle changingrequirements, risk management, resource allocation, and scalability? What arethe key trade-offs between flexibility, predictability, and speed of delivery in each model, and how can organizations determine which model is best suitedfor projects with varying levels of complexity, uncertainty, and stakeholder involvement?

Define functional requirements in the context of software engineering. Illustrate your answer with at least three clear examples from a typical online banking system.

Waterfall Model

A company in Kathmandu is developing a digital payment system that integrates banks, merchants, and customers. Identify and describe the key roles that contribute to the successful development and deployment of this project.

    -
  -
  - 14:00–16:00 **REVISION**
    - Revise Day 2 Architecture (1.1–1.9) and Day 2 Research (1.1–1.5).
  -
  - 16:30–19:00 **Data Structure and Algorithm Design**
    - *2. Advanced Data Structures and Algorithms*
      - 2.1. Advanced Trees: B and B+ Trees
      - 2.2. Advanced Graph: Planar Graphs and its applications in Geographic and Circuit Layout Design
      - 2.3. Network Flow Algorithms
      - 2.3.1. Ford-Fulkerson Algorithm
      - 2.3.2. Edmonds-Karp Algorithm

List the properties of B-tree. What are the disadvantages of B-tree? How are they resolved in B+ tree? Explain in detail.

B+ Tree

Construct an AVL tree for following sequence of elements: 5,10,15, 30, 25, 20, 4, 2,1.

Planar Graphs and its applications

Consider the following flow network: Vertices: {S, A, B, T} and Edges and capacities: (S, A, 10), (S, B, 10), (A, B, 2), (A, T, 8), (B,T, 9). Use the Ford-Fulkerson algorithm to find the maximum flow from source S to sink T.

Ford-Fulkerson algorithm to find the maximum flow from source S to sink T.[SN]

    -
  - 21:00–22:00 **SAME-DAY REVISION & ACTIVE RECALL**
    - Revise: SE 1.1–1.3 & DSA 2.1–2.3.2.

Consider the following flow network: Vertices: {S, A, B, T] and Edges and capacities: (S, A, 10), (S, B, 10), (A, B, 2), (A, T, 8), (B,
Use primary hash Ford-Fulkerson algorithm[SN]

-
- **DAY 4**
  - 05:00–08:00 **Operating System Design and Implementation**
    - *1. Operating System Architecture*
      - 1.4. Kernel Design and Implementation Strategies
      - 1.5. Case Studies : Linux Kernel Architecture, Windows NT Architecture
    -
    - *2. Advanced Process and Thread Management*
      - 2.1. Advanced Process Scheduling Techniques: Multi-core (Affinity based, Load Balancing, Cache and memory Optimization) and NUMA-aware scheduling
    -
  -
  - 14:00–16:00 **REVISION**
    - Revise Day 3 SE (1.1–1.3) and Day 3 DSA (2.1–2.3.2).
  -
  - 16:30–19:00 **Advanced Computer Architecture**
    - *2. Memory technology*
      - 2.1 Hierarchical Memory Structures
      - 2.2 Memory capacity planning
      - 2.3 Cache Memories and Management
      - 2.4 Virtual memory technology
      - 2.5 Page replacement algorithms
      - 2.5.1 FIFO algorithm
      - 2.5.2 Least recently used algorithms(LRU)
      - 2.5.3 Most frequently used algorithm (MFU)
    -
  -
  - 21:00–22:00 **SAME-DAY REVISION & ACTIVE RECALL**
    - Revise: OS 1.4, 1.5, 2.1 & Architecture 2.1–2.5.3.
  -

Advanced process and thread management

Compare and contrast advanced process scheduling techniques for multi-core systems, focusing on CPU affinity, load balancing, and NUMA-aware optimizations.

Differentiate between NUMA and UMA based multiprocessor architecture. In the context of NUMA architecture, how does NUMA aware scheduling enhance memory access?

Discuss various types of cache mapping with examples.

What new memory technoiogies can be used to improve hierarchical memory systems and make computational tasks faster and more efficient? Explain.

Draw the block diagram of direct mapping cache organization and explain it. Why cache is required in the computer?

Virtual memory

What are the essential factors to consider when designing and managing cache memory. Explain

Direct vs. associative mapping

Belady's Anomaly

What is page replacement and why is it essential? Compare the optimal page replacement algorithm with the FIFO algorithm using suitable examples.

How do FIFO, LRU, and LFU cache replacement policies differ in terms of their approaches and effectiveness? Discuss with examples.

Which page replacement algorithm works best for improving memory usage and system performance? Explain.

Briefly explain the key design considerations for cache memory and management.

You have been asked to design a cache with the following properties: Data words are 32 bits each; cache block contains 2048 bits; cache is direct mapped; address supplied from CPU is 32 bits; there are 2048 blocks in cache. Calculate (a) number of bits of offset, (b) number of bits of index, (c) number of bits in tag.

What are graphical data structures? You already have BST, AVL tree or B+ tree for storing and accessing data. If so, why do you need quadtree? Explain how the operations like search or insertion are more efficient in quadtree with a suitable example.

2.Days 5 to 8: Advanced Algorithms, Memory & Pipelining: Activating +4 and +7 day spaced repetition cycles.

- **DAY 5**
  - 05:00–08:00 **Data Structure and Algorithm Design**
    - *2. Advanced Data Structures and Algorithms*
      - 2.4. Advanced Shortest Path Algorithms
      - 2.4.1. Bellman-Ford Algorithm
      - 2.4.2. Johnson’s algorithm
      - 2.5. Graphical Data Structures: Quadtrees, KD Trees, R Trees
      - 2.6. Computational Geometry: Convex Hull and Voronoi Diagrams
    -
  -

Johnson's algorithm for all-pairs shortest path

Explain the working of Bellman-Ford algorithm with an example. When is the Bellman-Ford algorithm preferred over Dijkstra's?

Write an algorithm for Bellman-Ford algorithm. [SN]

Describe the insertion process in a R-tree. Provide a step-by-step breakdown with an example programming. set of points in a 2D plane.

Describe the insertion process in a Point KD-tree. Provide a step-by-step breakdown with an example set of points in a 2D plane.

Describe the insertion process in a Quadtree. Provide a step-by-step breakdown with an example set of points in a 2D plane.

How does Bellman-Ford algorithm differ from Dijkstra's algorithm for finding shortest path?[SN]

Define a convex hull. Explain how the Graham's Scan algorithm is used to find the convex hull of a set of points with a suitable example.

Compare and contrast different literature sources. Evaluate the effectiveness of systematic vs. narrative review methods in IT research.

How can conducting a literature review help in identifying a research gap? Choose a specific domain and illustrate the process of findingresearch gap with a relevant example.

- 14:00–16:00 🔄 **+4-DAY REVISION**
  - Revise Day 1: DSA (1.1–1.3) & OS (1.1–1.3). Focus on active recall.
- 16:30–19:00 **Research Techniques**
  - *2. Literature Review*
    - 2.1. Purpose of Literature Review
    - 2.2. Literature Resources and Techniques
    - 2.3. Conducting a literature review
    - 2.4. Referencing and Citation
    - 2.5. Evaluating literature review
    - 2.6. Identifying problem: Research question / problem statements and hypothesis
    - 2.7. Case Study: Prepare a review paper.

Develop two research titles, objectives, research questions, a problem statement and hypothesis on the topic "Role of AI in medical sciences". Also explain how can research gap be found in this particular topic.

Why literature review is important in research? How do youevaluate a literature review? Explain with examples.

Explain the difference between referencing and citation. Why is proper citation essential in academic research? Give examples.

Referencing

References

Why do we need DevOps-driven approach for the software development process? Explain the automated tools that contribute in the smooth deployments of the and operation of the software systems.

What is a problem statement in research

Citations and Referencing

- 21:00–22:00 **SAME-DAY REVISION & ACTIVE RECALL**
  - Revise: DSA 2.4–2.6 & Research 2.1–2.7.

Explain the core values and principles of the Agile Manifesto. How do they influence software development practices?

The Scrum process is widely adopted in modern Agile software development. Explain the various artifacts and ceremonies used in a Scrum-based process.

- **DAY 6**
- 05:00–08:00 **Software Engineering**
  - *2. Software Process Models and Project Management*
    - 2.1 Agile Development Principles: Manifesto for Agile Software Development, Benefits of Agile Methodologies
    - 2.2 Scrum and Kanban Methodologies: Scrum Roles, Artifacts, and Ceremonies, Kanban Boards and Workflows
    - 2.3 DevOps Culture and Principles: Overview of Jenkins, Docker, and Kubernetes
    - 2.4. Project Organization Concepts
      2.5Project Communication

Analyze the interplay between project organization (including roles, task assignment, and scheduling) and project communication (such as planned vs.unplanned communication, tools, and meetings) in software project management. How do these elements collectively contribute to the success or failure of a project, and what strategies can be employed to ensure they work jointly to achieve project goals? Provide example to support your analysis.

What is Docker, and how does it simplify the process of deploying and managing applications in different environments?

- 14:00–16:00 🔄 **+4-DAY REVISION**
  - Revise Day 2: Architecture (1.1–1.9) & Research (1.1–1.5).
- 16:30–19:00 **Operating System Design and Implementation**
  - *2. Advanced Process and Thread Management*
    - 2.2. Thread Libraries and Synchronization: Real-time (Semaphore-based, conditional variable, synchronization protocols) and Adaptive synchronization techniques (Adaptive locks, lock free data structure, synchronization protocol)
    - 2.3. Deadlock Detection Algorithms (Centralized, Distributed and Hierarchical)

A software company is facing frequent delays in product releases due to slow manual testing and poor collaboration between the development and QA teams. The company plans to implement DevOps practices with a focus on test automation, Continuous Integration (CI) using Jenkins, and containerization with Docker. Explain how these DevOps practices can help resolve the issues. What steps should the company take to implement them effectively?

Critically analyze real-time and adaptive thread synchronization mechanisms in modern OSes.

Compare and contrast semaphore-based ynchronization with conditional Variable based synchronization. Explain condition variable based thread synchronization with a real-world example.

- 21:00–22:00 **SAME-DAY REVISION & ACTIVE RECALL**
  - Revise: SE 2.1–2.3 & OS 2.2, 2.3.
- **DAY 7**

Evaluate centralized, distributed, and hierarchical deadlock detection algorithms in distributed systems

Derive speedup for normal pipelining. Differentiate super-pipelining, superscalar and normal pipelining.

Explain the concept of pipelining in computer architecture. How does pipelining improve the performance of a processor? Derive relation of speedup due to pipeline and discuss.

Explain six-stage instruction pipeline with suitable diagram.

What is vector processing? What are the different types of vector instructions? Explain.

Processor pipelining

Instruction Parallelism

Derive and explain the relation of speedup for normal pipelining. Differentiate super pipelining, superscalar pipelining and normal pipelining with example

If instruction processing is split into six phases, how would instruction pipelining be? Explain with diagram

- 05:00–08:00 **Advanced Computer Architecture**
  - *3. Principles of Pipelining and Vector Processing*
    - 3.1 Pipelining, Instruction and Arithmetic Pipelines
    - 3.2 Non-linear pipeline processor
    - 3.3 Pipeline processor classification
    - 3.4 Arithmetic pipeline design
    - 3.5 Instruction pipeline design

Pipelining doesn't decrease the time required for the execution of a single instruction; then how can it increase the overall throughput? Explain.

What are the linear and non-linear pipelines? Explain

What is arithmetic pipelining? With suitable examples and derivations describe how it speed up the computation?

Differentiate between research methodology, research design, and research methods. How do these components interact to ensure a coherent research study in IT?

Explain different research paradigms. How do these paradigms influence the choice of methodology in IT and computer systems research?

- 14:00–16:00 🔄 **+4-DAY REVISION**
  - Revise Day 3: SE (1.1–1.3) & DSA (2.1–2.3.2).

Why is research design important? Explain the stages involved in research design.

Compare and contrast experimental research, case study, and Simulation-based research in the context of solving problems.Discuss their advantages, limitations, and suitability for different types of research. Provide examples to support your analysis.

- 16:30–19:00 **Research Techniques**
  - *3. Research Methodology*
    - 3.1. Components of Research Methodology: Research problem, Objectives, Research Design, Data Collection, Sampling Techniques, Data Analysis, Hypothesis Formulation, Ethical Consideration and Validation and Reliability
    - 3.2. Research Design: Experimental, Exploratory, Descriptive and Explanatory
    - 3.3. Research Strategies: Experimental Research, Case Study, Action Research, Theoretical Research, Simulation based Research.
    - 3.4. Multidisciplinary Research Approaches

What is a problem statement in research, and how are research objectives and research design formulated? Explain their roles and interrelationships.

Examine different types of research design. Provide examples of their application in IT-related research problems?

Do you think hypothesis testing is necessary in computer engineering research?
How the hypothesis is formulated?
Explain Type-I and Type-II error.

Multidisciplinary approaches of research

What is an explanatory rescarch design? How does simulation-based research differ with experimental research in terms ofaccuracy and real-world applicability?

What is a research strategy? Explain the most commonly adopted research strategies in the field of computer engineering.

- 21:00–22:00 **SAME-DAY REVISION & ACTIVE RECALL**
  - Revise: Architecture 3.1–3.5 & Research 3.1–3.4.

Sampling and Non-sampling Errors

Suppose you are doing research on "ML-based house prize prediction." Develop a problem statement, research questions, objectives, and hypotheses for this study and justify your choices.

Consider a network design problem where you need to connect 5 cities with minimum cost. Use Kruskal's algorithm to find the Minimum Spanning Tree (MST) for the following edge weights: A-B: 4, A-C: 2, B-C: 3, B-D: 2, B-E: 3, C-D:4,C-E:5,D-E:1

DAY 8

- 05:00–08:00 **Data Structure and Algorithm Design**
  - *2. Advanced Data Structures and Algorithms*
    - 2.7. Case Studies in Red-Black Tree and Its Applications, Applications of Directed Acyclic Graph, Applications of Minimum Spanning Tree in Network Design
  - *3. Cache Efficient Data Structures*
    - 3.1. Memory Hierarchy and Cache Efficiency
    - 3.2. Principles of Cache-Efficient Data Structures
    - 3.3. Cache-Conscious Algorithms and Cache-Oblivious Algorithms

Use Kruskal's algorithm to find the Minimum Spanning Tree (MST) for the following graph: [SN]

What is memory hierarchy? How do the different hierarchies of memory help to enhance the performance of the computer? Explain.

What is cache memory? Explain the different cache replacement algorithms

Principles of Cache-Efficient Data Structures

Define cache-conscious algorithms. Explain the cache-conscious matrix multiplication in detail.

- 14:00–16:00 🔄 **+7-DAY & +4-DAY REVISION**
  - 2:00–2:40 (+4 Day): Day 4 OS (1.4–2.1) & Architecture (2.1–2.5.3).
  - 2:40–4:00 (+7 Day): Day 1 DSA (1.1–1.3) & OS (1.1–1.3).

What are cache-oblivious algorithms? Explain the cache-oblivious merge sort in detail.

What is the purpose of using different cache levels instead of using a single large cache memory? What are the principles of cache-efficient data structures? Explain in detail.

A software development company is building a hospital management system. The doctors want a feature that provides a detailed patient history, while the administrative staff wants a simple and quick access system. There is a conflict in their requirements.As a business analyst, how would you handle this situation?What steps would you take to balance stakeholder requirements while ensuring a feasible solution?

- 16:30–19:00 **Software Engineering**
  - *3. Requirements Engineering and Elicitation*
    - 3.1 Requirements Elicitation Concepts: Importance of Stakeholder Involvement, Challenges in Requirements Gathering
    - 3.2 Detailed Requirements Analysis: Completeness and Consistency, Clarity and Correctness, Realism and Verifiability, Traceability of Requirements

A software development team is tasked with building an online food deliveryinitial use cases for each scenario.... of all stakeholders.[SN]

Why is stakeholder involvement critical during the requirements gathering phase of a software project, and what are some common challenges faced in ensuring effective participation from all stakeholders? How can these challenges be addressed to ensure accurate and comprehensive requirements?

- 21:00–22:00 **SAME-DAY REVISION & ACTIVE RECALL**
  - Revise: DSA 2.7, 3.1–3.3 & SE 3.1, 3.2.

Why is stakeholder involvement crucial in requirements elicitation? Explain the challenges faced during the requirements gathering process and how they can impact the success of a software project.

3.Days 9 to 12: Advanced Systems & Data Collection: Deep dive into Distributed systems, Hashing, & Virtualization.

- **DAY 9**
  - 05:00–08:00 **Operating System Design and Implementation**
    - *2. Advanced Process and Thread Management*
      - 2.4. Distributed Mutual Exclusion Algorithms (Lamports, Token and Non Token Based Ricart Agarwala, Epidemic Algorithms, Maekawa's)
      - 2.5. Case Studies: Process Management in Linux and Windows.

Compare Lamport's, token-based, and Maekawa's algorithms for distributed mutual exclusion.

Explain Lamport's algorithm and Ricart-Agrawala's algorithm for distributed mutual exclusion. How do these algorithms ensure mutual exclusion in distributed systems? Discuss the key differences between the two algorithms and their use cases.

  - 14:00–16:00 🔄 **+4-DAY & +7-DAY REVISION**
    - 2:00–2:40 (+4 Day): Day 5 DSA (2.4–2.6) & Research (2.1–2.7).
    - 2:40–4:00 (+7 Day): Day 2 Architecture (1.1–1.9) & Research (1.1–1.5).

Explain various types of hazards in pipeline processor with example. Also, purpose solution for each type.

  - 16:30–19:00 **Advanced Computer Architecture**
    - *3. Principles of Pipelining and Vector Processing*
      - 3.6 Pipeline Hazards

Pipeline hazards

      - 3.7 Dynamic instruction scheduling

Tomasulo's Algorithm

      - 3.8 Vector instruction types

Scalar vs. vector pipeline

      - 3.9 Vector performance modeling

What are some common use cases of vector databases in the field of data science and AI? Explain.

  - 21:00–22:00 **SAME-DAY REVISION & ACTIVE RECALL**
    - Revise: OS 2.4, 2.5 & Architecture 3.6–3.9.
- **DAY 10**
  - 05:00–08:00 **Research Techniques**
    - *4. Data Collection*
      - 4.1. Primary and Secondary Data

Why is data collection essential in computer engineering related research, and what are the main methods used for collecting the data? Explain.

Briefly explain the methods used in data collection. What are the ethical challenges associated with survey-based data collection, and how can they be addressed?

- 4.2. Data Collection Methods: 4.2.1. Surveys, 4.2.2. Experiments, 4.2.3. Questionnaires, 4.2.4. Interviews, 4.2.5. Observation, 4.2.6. Experiments, 4.2.7. Documents

A researcher wants to measure student stress levels using a questionnaire.....Calculate Cronbach's... scale is reliable.[SN]

- 14:00–16:00 🔄 **+4-DAY & +7-DAY REVISION**
  - 2:00–2:40 (+4 Day): Day 6 SE (2.1–2.3) & OS (2.2–2.3).
  - 2:40–4:00 (+7 Day): Day 3 SE (1.1–1.3) & DSA (2.1–2.3.2).
- 16:30–19:00 **Software Engineering**
  - 3. *Requirements Engineering and Elicitation*
    - 3.3 Elicitation Activities: Identifying Actors and Stakeholders, Developing Scenarios and Use Cases, Refining Use Cases and User Stories, Identifying Relationships among Actors and Use Cases
    - 3.4 Managing Requirements: Maintaining Traceability, Negotiating Specifications with Clients, Documenting Requirements

A digital financial service platform requires users to complete identity verification through a Video KYC process before accessing its services. In this process, a user connects to the system via a live video session, presents identity documents, and interacts with a KYC officer or an automated verification system that performs document validation and liveness detection. Develop a use case description for the Video KYC process. Your answer should include the primary and secondary actors, preconditions, main flow of events alternative/exception flows, and postconditions

- 21:00–22:00 **SAME-DAY REVISION & ACTIVE RECALL**
  - Revise: Research 4.1–4.2.7 & SE 3.3, 3.4.

- **DAY 11**
  - 05:00–08:00 **Data Structure and Algorithm Design**
    - *3. Cache Efficient Data Structures*
      - 3.4. Profiling Tools for Analyzing Memory and Cache Efficiency
      - 3.5. Case Studies in Cache-Conscious B Tree and Cache Conscious Merge Sort, Cache-Oblivious merge sort, Profiling Tools: Valgrind and Cachegrind

Cache-Aware vs. Cache-Conscious algorithms

What are algorithm design techniques? Explain how the divide and conquer technique solves a problem with the reference to the merge sort. Also, analyse the time complexity of the merge sort.

Valgrind and Cachegrind

Describe the Bitonic Merge Sort with suitable example.

Store the following keys, K = (12, 14, 17, 21, 24, 27, 31, 34, 37} into hash table using perfect hashing. Use primary hash function h(k) = k % m where E = 10 and secondary hash function hui(k) = k % mj where mj =nj2.

What is universel hashing? Store the following keys,...in primary and second level hashing.[SN]

    - *4. Advanced Hashing Techniques*
      - 4.1. Hardware-Accelerated Hashing
      - 4.2. Perfect Hashing
      - 4.3. Cuckoo Hashing

Cuckoo Hashing

Store the following keys, K = {20, 33, 6, 45, 61, 11, 231, 90, 101} into the hash table using cuckoo hashing. Use hash function h1(k) = k % 11 and h2(k) = k % 13. Use both hash table size 15.

  - 14:00–16:00 🔄 **+4-DAY & +7-DAY REVISION**
    - 2:00–2:40 (+4 Day): Day 7 Architecture (3.1–3.5) & Research (3.1–3.4).
    - 2:40–4:00 (+7 Day): Day 4 OS (1.4–2.1) & Architecture (2.1–2.5.3).
  - 16:30–19:00 **Advanced Computer Architecture**
    - *3. Principles of Pipelining and Vector Processing*
      - 3.10 Vectorization
      - 3.11 Vectorizing compiler design

Consider two empty tables of size 5 each and two hash functions h1(k) = k mod 5 and h2(k) = (k/5) mod 5. Insert the keys 27, 2, 32 in this order using Cuckoo hashing and show the result.

What is vectorization? Discuss the challenges of vectorizing code and how a vectorizing compiler addresses these challenges.

What is array processing? Explain different types of array processors.

Array processor

- 3.12 Structures and Algorithms of Array Processors (SIMD Computers)
- 3.12.1 SIMD Array Processors, 3.12.2 SIMD Interconnection networks, 3.12.3 Parallel Algorithms for Array Processors, Algorithm examples – matrix multiplication

How can Cannon's algorithm and systolic arrays improve matrix multiplication efficiency in high-performance computing? Explain.

- 21:00–22:00 **SAME-DAY REVISION & ACTIVE RECALL**
  - Revise: DSA 3.4–4.3 & Architecture 3.10–3.12.3.

- **DAY 12**
  - 05:00–08:00 **Operating System Design and Implementation**
    - *3. Memory Management and Virtualization*

Memory management and virtualization

Compare and contrast Huge Pages and Transparent Huge Pages (THP) in modern operating systems.

Explain the core mechanisms of each algorithm, including their data structures and decision processes

- 3.1. Advanced Virtual Memory: Huge Pages, Transparent Huge Pages
- 3.2. Advanced Page Replacement Algorithms: Clock-pro and WSClock.
- 3.3. Cache-aware Memory Management
- 3.4. Principles for Virtualization
- 3.5. Hypervisor Architectures

Explain the concept of Transparent Huge Pages (THP) and its impact on system performance. How does it differ from traditional page management techniques? Discuss the scenarios where THP is most beneficial.

Explain ClockPro and WSClock page replacement algorithms. How do these algorithms improve upon traditional page replacementalgorithms like FIFO and LRU in terms of handling page faults and optimizing memory usage?

Hypervisor architectures

  - 14:00–16:00 🔄 **+4-DAY & +7-DAY REVISION**
    - 2:00–2:40 (+4 Day): Day 8 DSA (2.7, 3.1–3.3) & SE (3.1–3.2).
    - 2:40–4:00 (+7 Day): Day 5 DSA (2.4–2.6) & Research (2.1–2.7).
  - 16:30–19:00 **Research Techniques**
    - *4. Data Collection*

Why is sampling frame important in research? Explain about probability and non-probability sampling techniques.

How is sampling important in research? Explain the different types of probability and non-probability sampling techniques.

Critically assess the influence of different data sampling techniques on the accuracy, and generalizability of predictive models in research design.

      - 4.3. Data Preparation
      - 4.4. Data Sampling and Its Techniques
      - 4.5. Evaluation and Validation of Data

Types of quantitative data

    - *5. Data Analysis*

What is data coding? How do descriptive and inferential statistics complement each other in research analysis? Provide examples.

      - 5.1. Quantitative Data Analysis: 5.1.1. Types of Quantitative Data, 5.1.2. Data Coding, 5.1.3. Data Presentation: Tables, Bar charts, Scatter Graph, Line graph

Quantitative data analysis

Data Coding

Classify different types of data and variables. How do these classifications influence research design and analysis?

How does data preparation impact the reliability and validity of research outcomes? Critically analyse the role of data cleaning and transformation in ensuring data quality, minimizing bias, and cnhancing the robustness of research findings.

  - 21:00–22:00 **SAME-DAY REVISION & ACTIVE RECALL**
    - Revise: OS 3.1–3.5 & Research 4.3–5.1.3.

4.Days 13 to 16: Complex Analysis & System Design:OOAD, Statistical Analysis, & Multithreading.

- **DAY 13**
  - 05:00–08:00 **Software Engineering**

Using the Library Management System scenario, analyze how class diagrams and sequence diagrams can be used to model the system's structure and behavior. How do these diagrams help in understanding the relationships between classes and the flow of interactions during key processes like borrowing a book? What challenges might arise when creating these diagrams, and how can they be resolved to ensure accurate system design?

Encapsulation

A software development team is designing an Online Shopping System where customers can browse products, add.....Provide examples to illustrate your answer.[SN]

What is inheritance? Explain the significance of inheritance of in software development process?

- *4. Object-Oriented Analysis and Design with UML*
  - 4.1 OOAD Principles: Encapsulation, Abstraction, Inheritance, Polymorphism
  - 4.2 Analysis Object Models and Dynamic Models: Entity, Boundary, and Control Objects, Generalization and Specialization
  - 4.3 UML Diagrams: Use Case Diagrams, Class Diagrams, Sequence Diagrams, Activity Diagrams, State Machine Diagrams

Discuss the four fundamental principles of Object-Oriented Analysis and Design (OOAD): ... the design process.[SN]

Sort the following list of numbers using Radix Sort: [170, 45, 75, 90, 8,802, 24, 2, 66]. Show the steps for each pass and the final sorted list. How can Radix Sort be made to work for a list of names (strings) of different lengths?

14:00–16:00 🔄 **+4-DAY & +7-DAY REVISION**

A bank wants to...Draw a Use Case Diagram,,,used in the design process.[SN]

- 2:00–2:40 (+4 Day): Day 9 OS (2.4–2.5) & Architecture (3.6–3.9).
- 2:40–4:00 (+7 Day): Day 6 SE (2.1–2.3) & OS (2.2–2.3).

16:30–19:00 **Data Structure and Algorithm Design**

Explain why Shortest Interval may fail to give the optimal solution.

- *4. Advanced Hashing Techniques*
  - 4.4. Hashing in Distributed Systems
  - 4.5. Case Studies in GPUs and FPGAs

Describe the Odd-Even Transposition Sort algorithm and how it alternates between even and odd phases to sort elements.

- *5. Advanced Sorting Algorithms*
  - 5.1. Non-comparison-based sorting: Radix sort, bucket sort
  - 5.2. Parallel Sorting Algorithms (multi-core and distributed systems)
  - 5.3. External sorting and its applications in large-scale data engineering

Sort the following numbers using Radix Sort: 150, 35, 45, 80, 505, 24, 2, 76. Show the steps for each pass and the final sorted list. How can Radix Sort be made to work for a list of names (strings) of different lengths?

Explain the working of Counting Sort with an example. Provide an example where Counting Sort performs poorly due to a large range of values.

Explain the working of Counting Sort with an example. Derive..... Counting Sort performs poorly due to a large range of values.[SN]

Too many parallel threads in sorting algorithm degrade the algorithm's performance. How do you resolve this? Explain with the example of parallel quick sort.

21:00–22:00 **SAME-DAY REVISION & ACTIVE RECALL**

- Revise: SE 4.1–4.3 & DSA 4.4–5.3.

DAY 14

What is the primary purpose of latency hiding techniques in multithreaded architecture, and how do these techniques improve overall system performance?

Explain Multi-Threading along with its types, models, advantages and disadvantages.

05:00–08:00 **Advanced Computer Architecture**

- *4. Multithreaded architecture*
  - 4.1 Latency hiding techniques
  - 4.2 Multithreading principles
  - 4.3 Multithreaded architecture
  - 4.4 Cluster computing
  - 4.5 Neural Computing
  - 4.6 Grid computing
  - 4.7 Comparison of Cluster computing and Grid computing
  - 4.8 Advantages and disadvantages of Grids

What is a multithreading architecture? Explain the principles used in multithreading architecture to increase the throughput of a computer.

Describe the principles of thread synchronization in multithreaded systems. What are mutexes, semaphores, and barriers?

Compare the cluster computing with grid computing showing similarities as well as differences.

Explain the key differences between cluster computing, neural computing, and grid computing. How does each approach contribute to high-performance computing, and in what scenarios would one be preferred over the others?

Discuss cluster computing along with its performance challenges, pros and cons.

14:00–16:00 🔄 **+4-DAY & +7-DAY REVISION**

- 2:00–2:40 (+4 Day): Day 10 Research (4.1–4.2) & SE (3.3–3.4).

What is time series data? Explain the basic techniques used in time series analysis.

A group of 6 employees completed a training.. perform this test.[SN]

How is the t-test different from the z-test? Explain ANOVA and its significance in research.

A cloud service provider..ANOVA..their significance for storage selection[SN].

- 2:40–4:00 (+7 Day): Day 7 Architecture (3.1–3.5) & Research (3.1–3.4).

Why is the Chi-Square test important in research? Explain the Chi-square test with an example.

Describe the role of multivariate analysis in understanding complex IT data, including server performance indicators, user activity, and transaction records. Further, discuss how techniques such as Principal Component Analysis (PCA) help reduce dimensionality and make the results easier to interpret

Eritically evaluate the role of descriptive and inferential statistics in quantitative research. How do these methods contribute to hypothesis testing and decision-making?

- 16:30–19:00 **Research Techniques**
  - *5. Data Analysis*
    - 5.1.4. Analysis Techniques: Descriptive Statistics, Inferential Statistics (Z-test, t-test, Chi-square test, ANOVA), Regression Analysis, Time Series Analysis and Multivariate Analysis
    - 5.2. Qualitative Data Analysis: 5.2.1. Qualitative Data, 5.2.2. Analyzing Textual Data, 5.2.3. Analyzing Non-Textual Data

Regression

Chi square

Describe Grounded Theory, Distributed Cognition, and Activity Theory, and explain their significance in qualitative analysis.

How is regression analysis important in research? Briefly explain simple linear and multivariate regression.

Qualitative Data Analysis

A researcher uses.... Type I and Type II errors... State the null and alternative hypotheses?[SN]

- 21:00–22:00 **SAME-DAY REVISION & ACTIVE RECALL**
  - Revise: Architecture 4.1–4.8 & Research 5.1.4–5.2.3.

How do you analyze textual and non-textual data? Explain briefly.

Differentiate between textual and non-textual qualitative data. Discuss the role of coding and thematic analysis in extracting meaningful insights from IT user feedback or software bug reports.

Briefly explain the basic techniques used for data presentation and analysis.

- **DAY 15**
  - 05:00–08:00 **Operating System Design and Implementation**
    - *3. Memory Management and Virtualization*
      - 3.6. Virtualization of Memory and I/O in Multi-Core Systems
      - 3.7. Memory Isolation and Protection in Virtualized Environments.
      - 3.8. Case studies: Memory Management in Linux, Virtualization Technologies (ZEN, VMware and AMD V)

Evaluate the security implications of different memory virtualization approaches.

Explain hardware-assisted virtualization technologies (AMD-V, Intel VT-x) and their impact.

File System and Storage Management

    - *4. File System and Storage Management*
      - 4.1. Distributed File Systems: HDFS, Ceph, Google File System
      - 4.2. Storage Technologies: SSDs, NVM, Persistent Memory.
      - 4.3. Modern File Systems: ZFS, Btrfs, EXT4, APFS.

Evaluate modern storage technologies-SSDs, NVMe, and persistent memory (PMEM)-focusing on performance, durability, and OS integration challenges.

Critically analyze ZFS, Btrfs, EXT4, and APFS in terms of scalability, data integrity, and advanced features.

What is a file? How does the OS identify the type of files? Discuss different methods used by the Operating System to keep track of which disk blocks go with which file.

  - 14:00–16:00 🔄 **+4-DAY & +7-DAY REVISION**
    - 2:00–2:40 (+4 Day): Day 11 DSA (3.4–4.3) & Architecture (3.10–3.12.3).
    - 2:40–4:00 (+7 Day): Day 8 DSA (2.7, 3.1–3.3) & SE (3.1–3.2).
  - 16:30–19:00 **Software Engineering**
    - *4. Object-Oriented Analysis and Design with UML*
      - 4.4 Modeling Interactions and Object Lifecycles: Mapping Use Cases to Objects, Modeling Associations and Aggregations, Modeling Inheritance Relationships
      - 4.5 Reviewing Analysis and Design Models: Iterative Refinement, Client Sign-Off Procedures
      - 4.6 Case Study: Applying OOAD and UML to a Real-World Application

Formal Technical Review (FTR)

    - *5. Design Patterns and Reuse*

Compare and contrast Specification Inheritance and Implementation Inheritance. Provide examples to illustrate their applications and explain how Delegation and the Liskoy Substitution Principle relate to these concepts.

Composite Pattern

- 5.1 Reuse Concepts: Solution Objects and Reusable Components, Specification vs. Implementation Inheritance, Delegation and the Liskov Substitution Principle
- 5.2 Common Design Patterns: Creational Patterns: Factory, Abstract Factory; Structural Patterns: Adapter, Composite, Bridge; Behavioral Patterns: Strategy, Command

What is a design pattern in software? Explain any two design patterns.

What is the role of tabulation in dynamic programming? Explain how it improves the performance of an iterative algorithm with a suitable example.

- 21:00–22:00 **SAME-DAY REVISION & ACTIVE RECALL**
  - Revise: OS 3.6–4.3 & SE 4.4–5.2.

Explain the Interval Scheduling problem. Describe the Shortest Interval and Fewest Conflicts greedy strategies with an example.

Given the following set of characters and their frequencies, construct a Huffman tree and derive the Huffman codes for each character: A: 5, B: 9, C: 12, D: 13, E: 16, F: 45. Show the steps.

- **DAY 16**
- 05:00–08:00 **Data Structure and Algorithm Design**
  - *6. Dynamic Programming and Greedy Algorithms*
  - 6.1. Engineering applications of dynamic programming: Resource Scheduling
  - 6.2. Greedy algorithms: Huffman coding and its relevance to data compression in embedded systems

Embedded System and static Huffman Coding

  - 6.3. Optimizing dynamic programming techniques for real time systems

Explain the concept of dynamic programming. How does it differ from greedy algorithms? Provide an example to demonstrate how a problem is formulated and solved using dynamic programming.

Given the following set of characters and their frequencies, construct a Huffman tree and derive the Huffman codes for each character: A: 7, B: 3, C: 15, D: 9, E: 16, F: 50. Show the steps.

- 14:00–16:00 🔄 **+4-DAY & +7-DAY REVISION**
  - 2:00–2:40 (+4 Day): Day 12 OS (3.1–3.5) & Research (4.3–5.1.3).
  - 2:40–4:00 (+7 Day): Day 9 OS (2.4–2.5) & Architecture (3.6–3.9).

Explain the concept of dynamic programming. How does it differ from greedy algorithms? Provide an example to demonstrate how a problem is formulated and solved using dynamic programming.

- 16:30–19:00 **Research Techniques**

Importance of data visualization

  - *5. Data Analysis*
    - 5.3. A detailed Case Study in data visualization and analysis using Python libraries: Pandas, NumPy, Matplotlib, Seaborn, SciPy, Scikit-learn

What is the significance of data visualization in quantitative analysis, and how do Matplotlib and Scikit-learn contribute to effective data presentation and interpretation decision-making in research.

  - *6. Ethical Issues in Research*

Ethical Issues in Research

Research Ethics

    - 6.1. Right of People directly involved, 6.2. Responsibilities of an Ethical Researcher, 6.3. Data Privacy and Security, 6.4. Plagiarism and Intellectual Property, 6.5. Algorithmic Bias and Fairness, 6.6. Environmental Responsibility, 6.7. Conflict of Interest, 6.8. Copyright and Patent, 6.9. ACM Code of Ethics and IEEE Code of Ethics

Describe about importance of plagiarism, conflict of interest, patent and codes of ethics in research.

As an early-stage researcher, how do you write and present your research? Explain.

- 21:00–22:00 **SAME-DAY REVISION & ACTIVE RECALL**
  - Revise: DSA Unit 6 & Research 5.3, Unit 6.

What do you mean by data privacy and copyright? How can researchers maintain objectivity when their research is funded by private corporations with vested interests?

5.Days 17 to 20: Completion & Exam Simulation: Syllabus finalization, Multicore, Testing, & Full Past Papers.

- **DAY 17**
  - 05:00–08:00 **Software Engineering**
    - *5. Design Patterns and Reuse*
      - 5.3 Selecting and Applying Design Patterns: Heuristics for Pattern Selection, Integrating Patterns into Designs
      - 5.4 Managing Reuse: Documenting Reuse Strategies, Assigning Responsibilities for Reusable Components
    - *6. Software Quality Assurance and Testing*
      - 6.1 Software Quality Concepts and Attributes: Reliability, Usability, Efficiency, Maintainability, Software Safety and Security
      - 6.2 Software Cost Estimation: Introduction to the COCOMO Model, Estimation Techniques and Tools
      - 6.3 Software Quality Assurance Planning and Process: Quality Assurance Activities, Integrating Testing Tools with CI/CD Pipelines

Why is software testing essential in the software development life cycle? Describe how software testing is carried out, and explain various types of testing approaches with examples, based on different needs and purposes.

Compare and contrast HDFS and GFS. Discuss their architectures,use cases, and how they handle scalability and consistency challenges in distributed environments.

  - 14:00–16:00 🔄 **+4-DAY & +7-DAY REVISION**
    - 2:00–2:40 (+4 Day): Day 13 SE (4.1–4.3) & DSA (4.4–5.3).
    - 2:40–4:00 (+7 Day): Day 10 Research (4.1–4.2) & SE (3.3–3.4).
  - 16:30–19:00 **Operating System Design and Implementation**
    - *4. File System and Storage Management*
      - 4.4. Scalability and Consistency Challenges in file and storage..
      - 4.5. Case Studies: DNA Storage, Holographic Storage, Quantum Storage

Discuss the trade-offs between scalability and consistency in distributed file systems, with examples from industry solutions.

Assess DNA storage, holographic storage, and quantum storage as next-generation solutions, addressing their principles, challenges, and potential applications.

    - *5. Security and Isolation in Operating Systems*
      - 5.1. Kernel Security Mechanisms: SELinux, AppArmor.
      - 5.2. Isolation Techniques: Namespaces, Cgroups, and Sandboxing.
      - 5.3. Address Space Layout Randomization (ASLR) and Stack Protection.
      - 5.4. Mitigation Techniques for Side-channel Attacks.
      - 5.5. Case Studies: Linux Security Module, Android Security Architecture, macOS Security Architecture)

What is SELinux (Security-Enhanced Linux), and how does it improve security in a Linux-based operating system? What are the four main components of SELinux and write the role of each component.

Mitigation techniques for side channel attack

  - 21:00–22:00 **SAME-DAY REVISION & ACTIVE RECALL**
    - Revise: SE 5.3–6.3 & OS 4.4–5.5.
- **DAY 18**

Explain the role of POSUX threads (PThreads) in shared memory systems. How does PThread synchronization (mutexes, condition variables) ensure safe and efficient parallel execution in a multiprocessor environment?

Briefly describe the different cache coherence protocols and their role in maintaining data consistency.

Explain cache coherence problem and its solutions.

- 05:00–08:00 **Advanced Computer Architecture**
  - *5. Multicore Processor Design*
    - 5.1. Introduction to Multicore Processors, 5.2. Design Principles, 5.3. Memory Hierarchy, 5.4. Parallel Programming, 5.5. Challenges, 5.6. GPU architectures and GPGPU computing

What are the different levels of cache memories used in multi-core computers and why are they used? Explain.

What are some of the use cases of vector database in advanced computing. Explain.

What is a multiprocessor operating system? Discuss briefly on the classification of multiprocessor operating systems.

Parallel Virtual Machine

Compare and contrast Symmetric Multiprocessing (SMP) and Asymmetric Multiprocessing (AMP) in terms of process scheduling, resource sharing, and fault tolerance. Provide examples of real-world implementations

Describe the working principle of PVM (Parallel Virtual Machine) as a distributed scheduler. How does PVM manage task allocation .Illustrate with a practical scenario.

  - *6. Multiprocessor Operating System*
    - 6.1 Classification of Multiprocessor OS, 6.2 Software requirements, 6.3 Requirement of implementation, 6.4 Distributed scheduler – PVM, 6.5 PThread in shared memory systems
- 14:00–16:00 🔄 **+4-DAY & +7-DAY REVISION**
  - 2:00–2:40 (+4 Day): Day 14 Architecture (Unit 4) & Research (5.1.4–5.2.3).
  - 2:40–4:00 (+7 Day): Day 11 DSA (3.4–4.3) & Architecture (3.10–3.12.3).
- 16:30–19:00 **Operating System Design and Implementation**
  - *6. Real-Time and Embedded Operating Systems*
    - 6.1. Real-Time Scheduling Algorithms (EDF, RMS, RR, LLF), 6.2. Embedded OS Features: TinyOS, FreeRTOS, 6.3. Energy-efficient OS Design, 6.4. Case Study: Android and Embedded Linux

What is a Real-Time Operaing System (RTOS), and how does it differ from a general-purpose operating system (GPOS)? List feature of embedded operating system.

  - *7. Distributed and Cloud Operating Systems*

Kubernetes

Discuss different types of transparency in DOS. For each type, explain how it enhances user experience and system efficiency. What challenges might system designers face when trying to achieve these transparencies in large scale distributed system

    - 7.1. Concepts (Transparency, Communication, Synchronization), 7.2. Cloud OS: Kubernetes, OpenStack, 7.3. Fault Tolerance and Load balancing, 7.4. Resource Management, 7.5. Case studies: Amazon DynamoDB, Microsoft Azure
- 21:00–22:00 **SAME-DAY REVISION & ACTIVE RECALL**
  - Revise: Architecture Units 5–6 & OS Units 6–7.

- **DAY 19**

Test Cases, Test Stubs, and Drivers

- 05:00–08:00 **Software Engineering** *(Final Syllabus Modules)*
  - *6. Software Quality Assurance and Testing*
    - 6.4 Testing Concepts: Faults, Erroneous States, and Failures, Test Cases, Test Stubs, and Drivers

What are different types of software testing. Explain the black-box testing and white-box testing.

    - 6.5 Testing Activities: Unit Testing, Integration Testing, System Testing, Regression Testing
    - 6.6 Automated Testing Frameworks: Introduction to Tools like Selenium and JUnit

Unit Testing

What is a software configuration item (CI)? How are configuration items managed throughout the software development lifecycle? Explain the importance and key objectives of configuration management in software engineering.

CASE Tools for Configuration Management

- *7. Configuration Management and DevOps Tools*
  - 7.1 Configuration Management Planning, 7.2 Change Management, 7.3 System Building, 7.4 CASE Tools for Configuration Management (Git, Subversion), 7.5 DevOps Tools in Configuration Management (Jenkins, Docker, Kubernetes)

What is Change Management in software development? Explain the key steps involved in managing change requests, and discuss how version and release management ensure software stability.

- 14:00–16:00 🔥 **PAST PAPER 1**
  - Attempt Past Paper 1 under full exam conditions without notes.
- 16:30–19:00 **Software Engineering**
  - *8. Emerging Trends in Software Engineering (Mapped from Unit 8 / X)*
    - 8.1.1 Cloud-Native Development: Microservices Architecture, Serverless Computing
    - 8.1.2 AI in Software Engineering: AI-Assisted Coding and Testing, Machine Learning Applications
    - 8.1.3 Software for IoT and Edge Computing
    - 8.1.4 Sustainability in Software Engineering: Eco-Friendly Practices, Green Coding
    - 8.1.5 Secure Software Development Practices: Writing Secure Code, Common Web Security Risks

Discuss the role of AI in Software Engineering. How do AI-assisted coding and testing tools improve development processes? Provide examples of tools and their impact.

Discuss the role of AI in Software Engineering. How do AI-assisted coding and testing tools improve development processes? Provide examples of tools and their impact.

AI in Software Engineering

- 21:00–22:00 **PAST PAPER 1 REVIEW**
  - Analyze mistakes from Past Paper 1. Mark topics as 🔴 (Cannot answer), 🟡 (Partially know), 🟢 (Can answer).

- **DAY 20**
  - 05:00–08:00 **FINAL SYLLABUS SYNTHESIS & DIAGRAM REFRESH**
    - Review high-yield diagrams and algorithms across all subjects (e.g., Red-Black Tree rotations, Vectorization pipelines, UML Sequence diagrams, Process scheduling state machines).
  - 14:00–16:00 🔥 **PAST PAPER 2**
    - Attempt Past Paper 2 without notes under strict timed conditions.
  - 16:30–19:00 **targeted RED-LIST REPAIR**
    - Do not study any new topics. Take the 🔴 red-list from Past Paper 1 and Past Paper 2. Revise strictly:
      - Questions you could not answer
      - Algorithms or pseudocode you forgot
      - Formulas and time complexities you missed
      - Diagrams you could not draw

- 21:00–22:00 **FINAL DEFENSE REVISION**
  - Final quick sweep of weak definitions, formulas, and critical concepts. Rest early.
- Research Techniques
- 7.. Presentation of the Research

How does the choice of research presentation format (written, visual, or oral) impact the effectiveness of conveying complex technical
-information? Provide examples.

As an early-stage researcher, how do you write and present your research? Explain.

Explain the essential qualities of a good research report. Outline and describe the critical steps in developing a research proposal.

Based on the passage below, answer the following questions.
Researchers tracked thousands of look-alike domain names and fake websites (around 2,600 websites surfaced between December I, 2024 and February 3. 2025).....Formulate four research hypotheses from the research questions.